DATA 260 Homework 6 Report

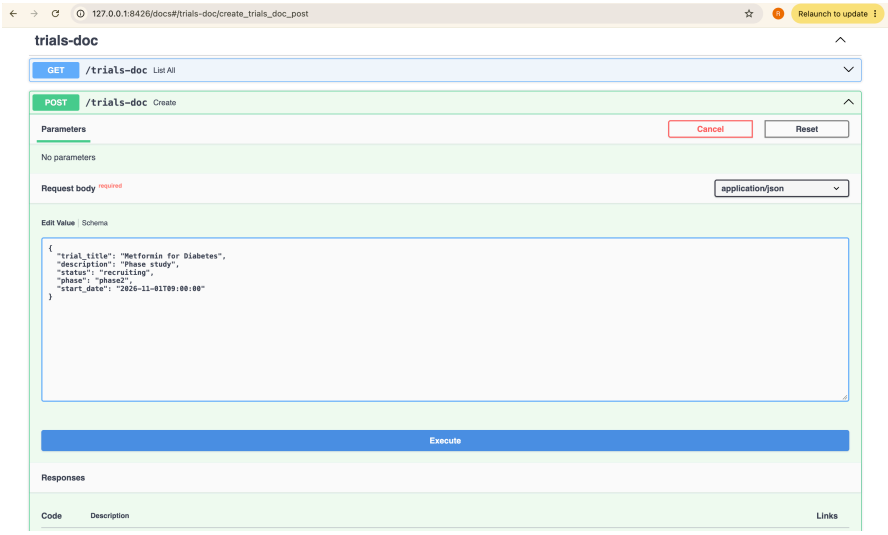
GITHUB REPOSITORY: https://github.com/rutujasaste06/data260-5825
Student ID- 020065825
SID4- 5825
PORT_BASE - 8425
PREFIX - s5825
SEED - 5825
VERIFY_SEED – 265825
DOMAIN_ID - 1
DOMAIN - Clinical trial listings
HARDWARE - MacBook Pro, Apple M5 chip
LOCAL MODEL USED: qwen3:8b (Ollama)
TAGGED COMMIT HASH: 3f4af3df9e9a320556c14953eef39afb3c5e8078

Outputs:

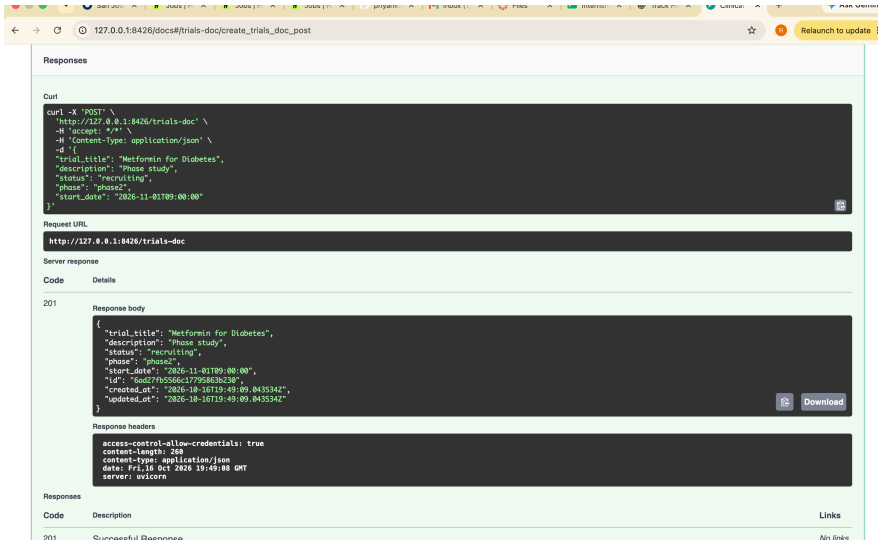
Part 1 - MongoDB Document API:

I. **Database:**

POST: Create the trial-doc

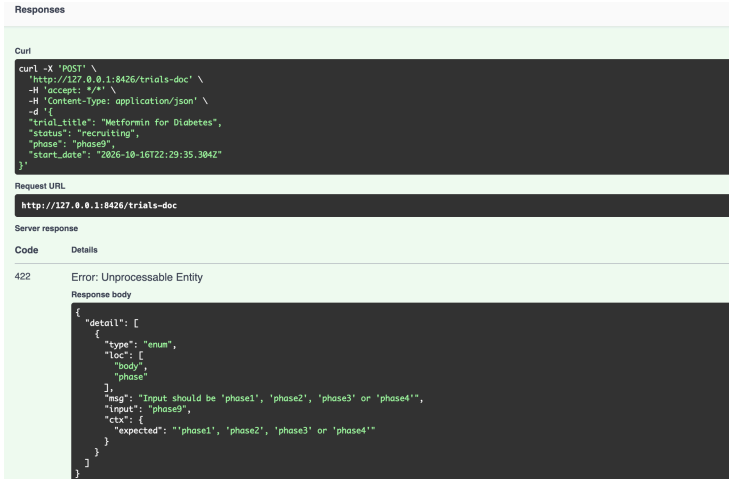


Returns 201 output

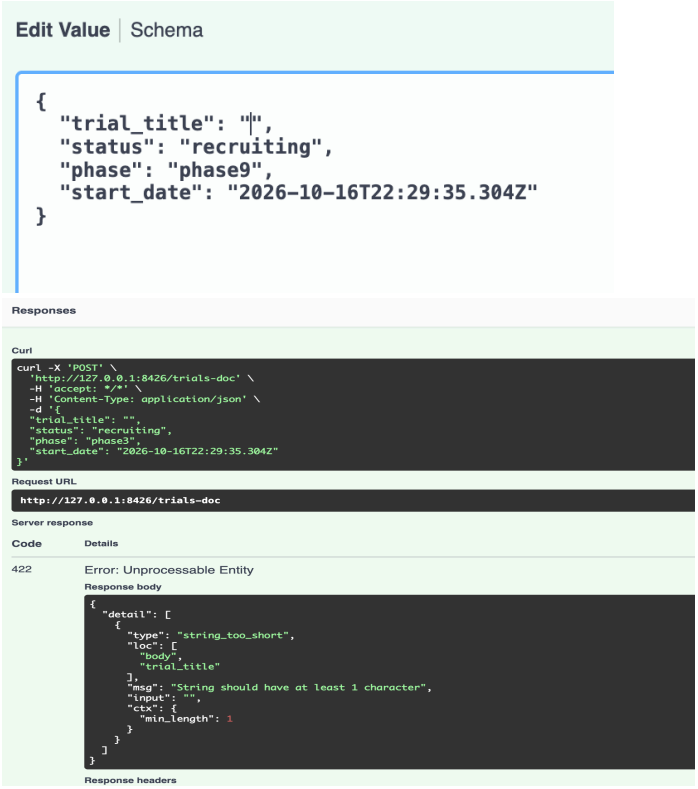


POST: Adding phase as phase9 which is out of the 4 defined phases this gives an error 422 with error message.

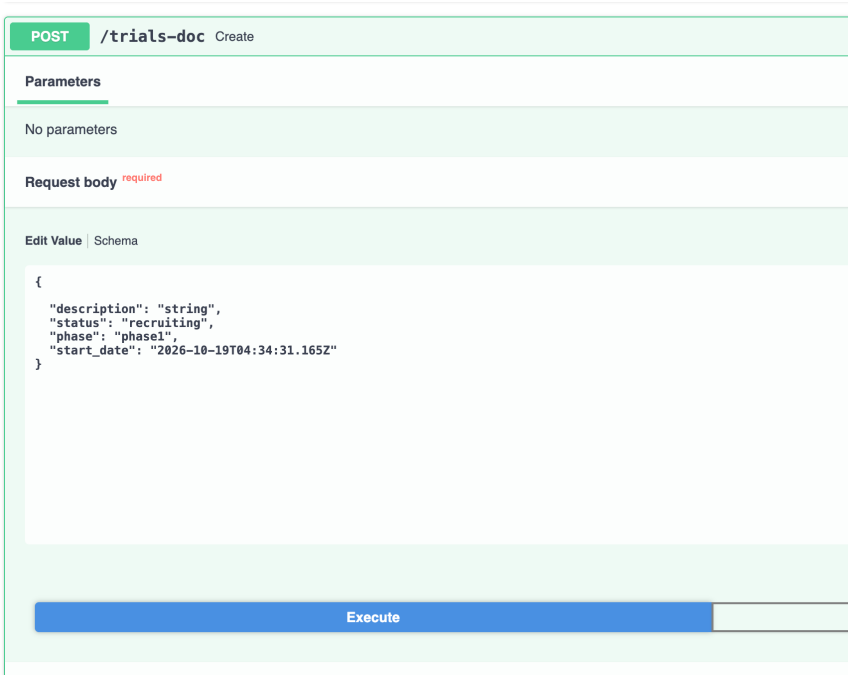




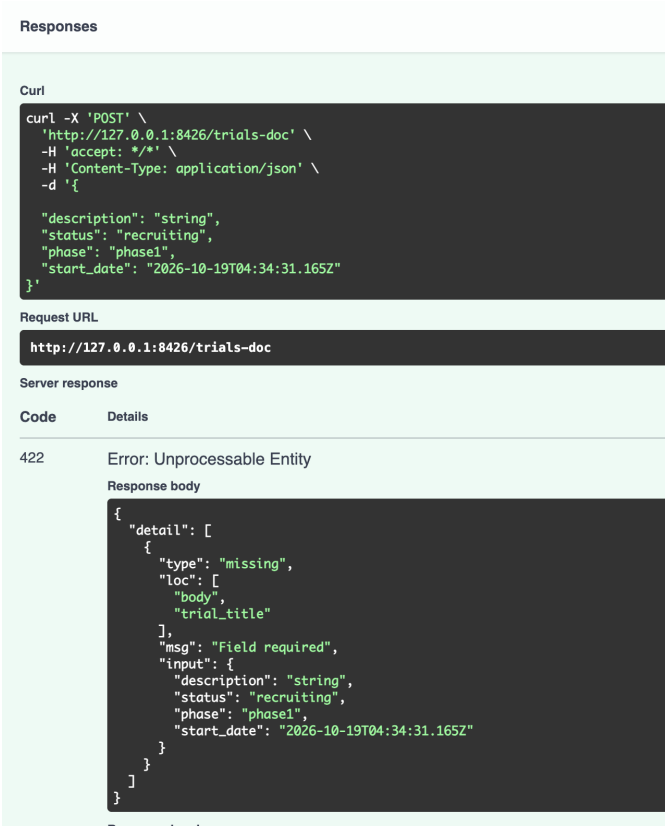
POST with trial_title missing: Gives an error response as string should have atleast 1 character



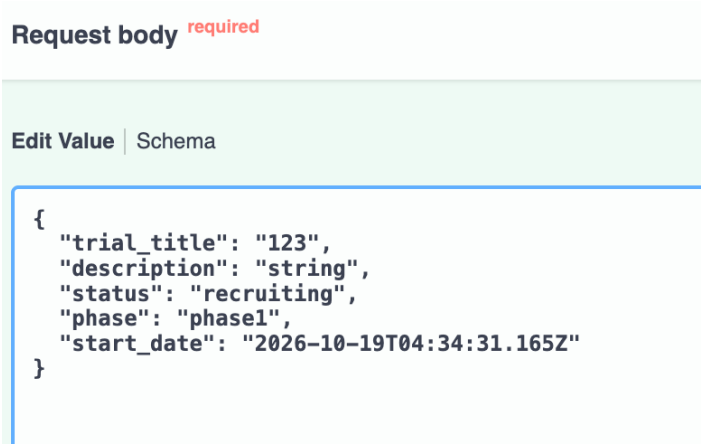
POST with complete trial_title missing field; Gives an error response as error code 422



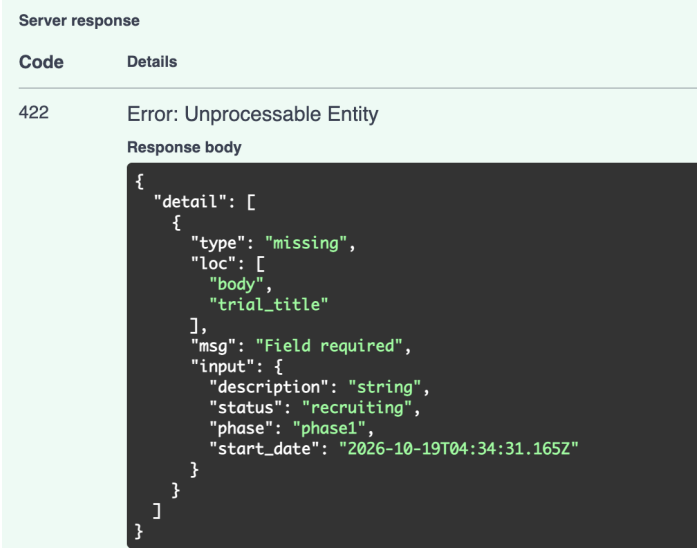
Response:



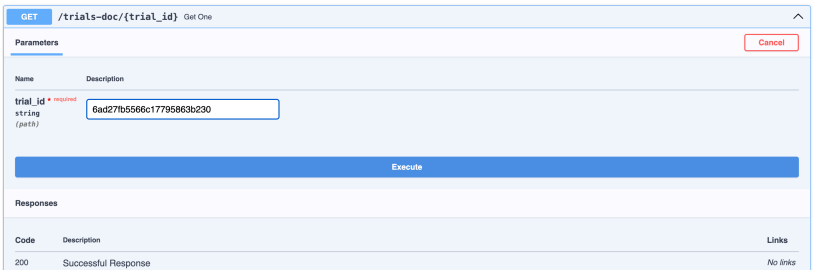
POST with invalid datatype in title field



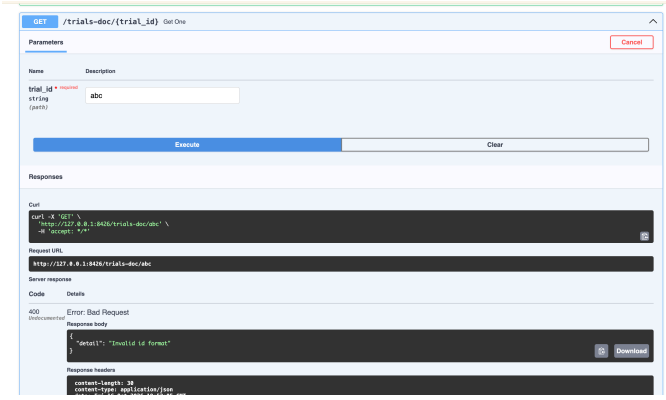
Response: Gives 422 error



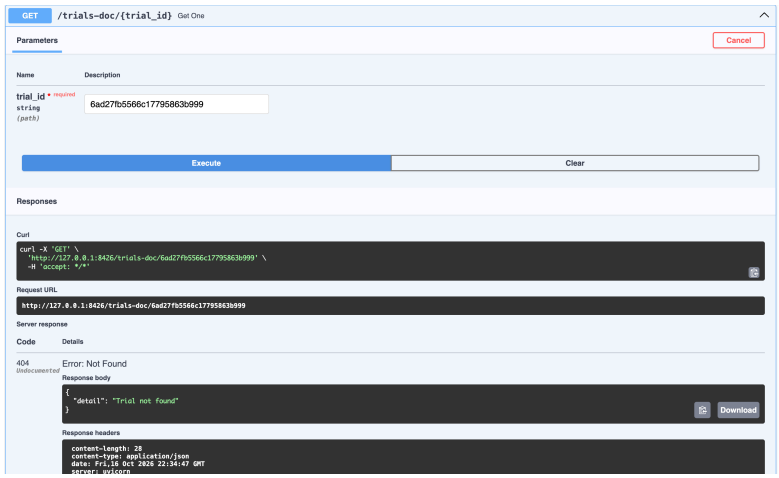
GET: Get trial doc as per the id



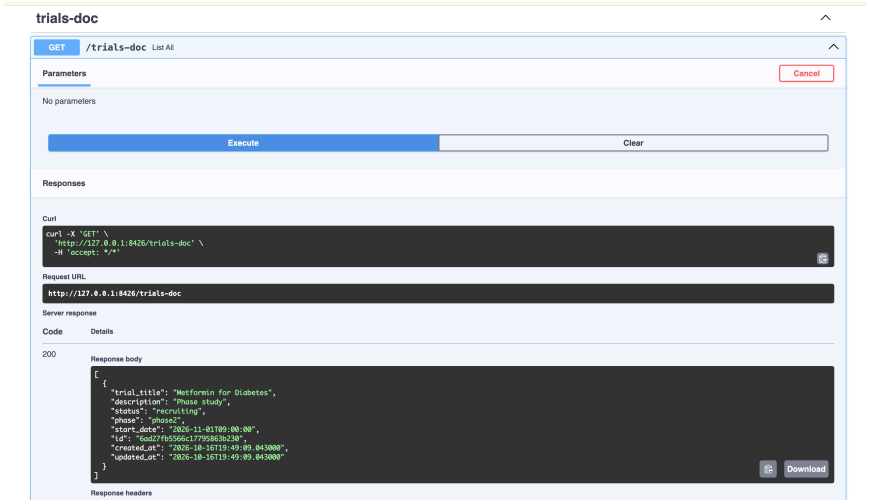
GET: Invalid id will give 400 errror with invalid id format



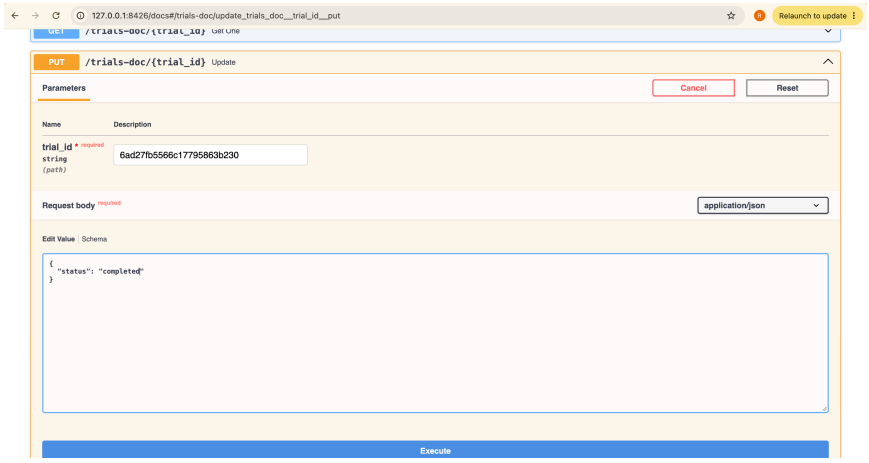
GET: Using valid looking id that doesnot exist gives 404 not found error

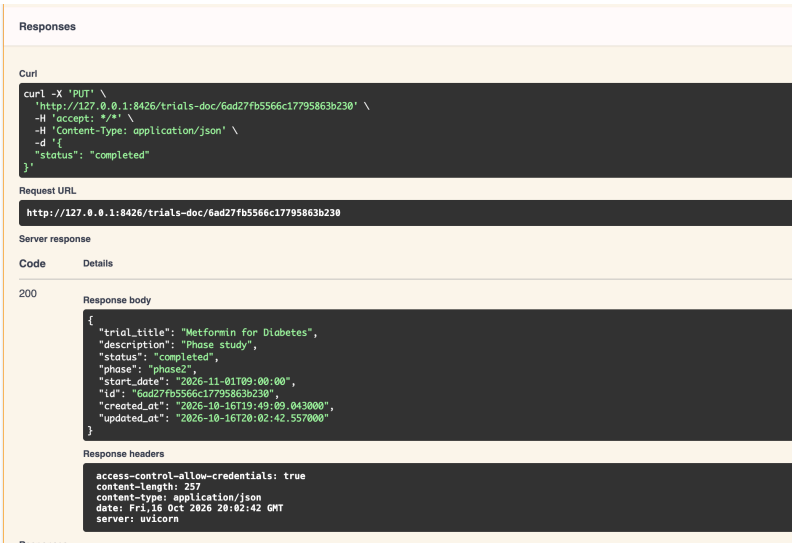


GET all: Get all trials doc

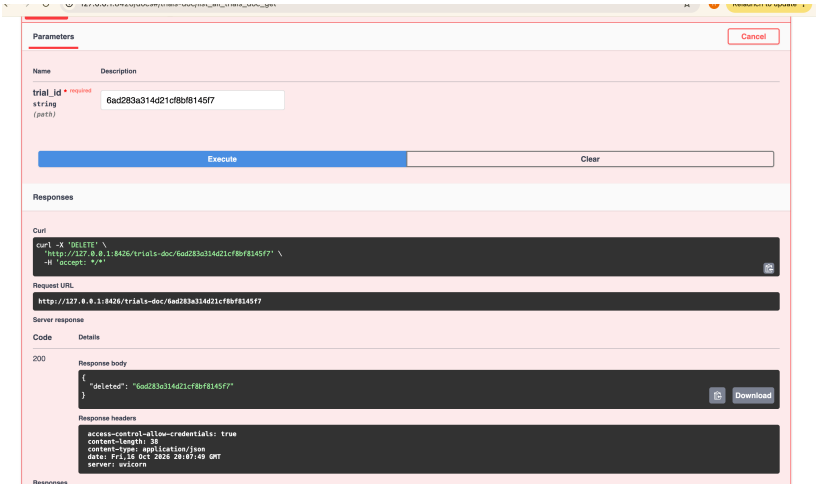


PUT: Update the trial doc with status as completed.

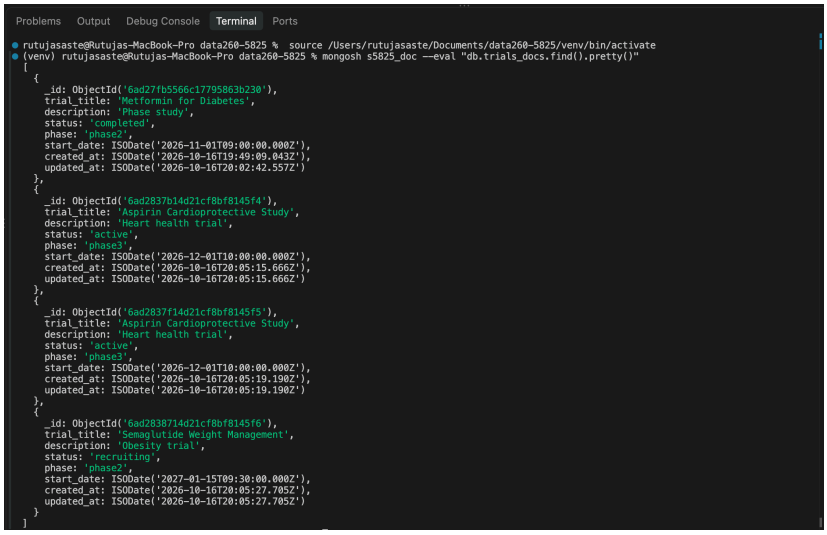




DELETE: Delte the trial doc using ID



Mongo database proof showing all trial id doc in bson format.



Pydantic model that checks the data before it reaches MongoDB (this is what gives the 422 errors)

```python
class TrialIn(BaseModel):
    trial_title: str = Field(min_length=1, max_length=100)
    description: Optional[str] = None
    status: Status = Status.recruiting
    phase: Phase
    start_date: datetime
```

MongoDB's own rule for what a document must look like

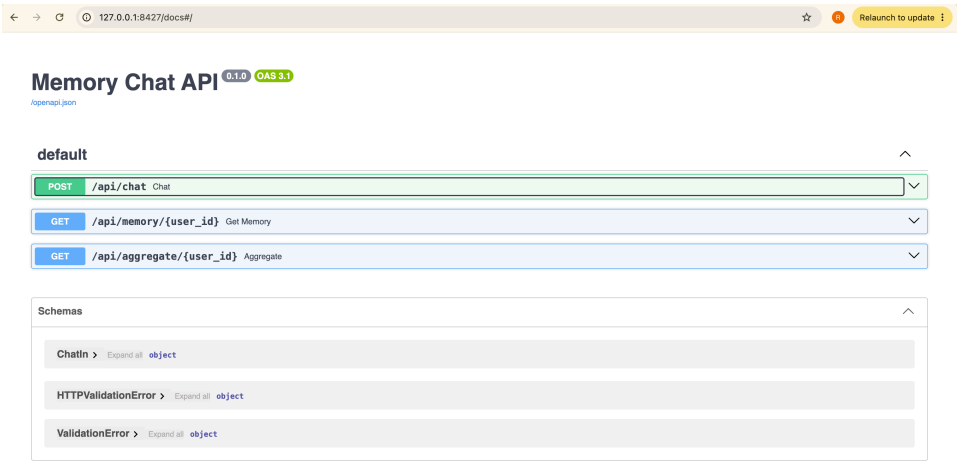
```python
VALIDATOR = {"$jsonSchema": {
    "bsonType": "object",
    "required": ["trial_title", "status", "phase", "start_date", "created_at", "updated_at"],
    "properties": {
        "trial_title": {"bsonType": "string", "maxLength": 100},
        "description": {"bsonType": "string"},
        "status": {"enum": STATUSES},
        "phase": {"enum": PHASES},
        "start_date": {"bsonType": "date"},
        "created_at": {"bsonType": "date"},
        "updated_at": {"bsonType": "date"},
    }}}
```

The create route. It saves the document and returns 201.

```python
@router.post("", response_model=TrialOut, status_code=201)
def create(payload: TrialIn):
    doc = payload.model_dump(mode="python", exclude_none=True)
    doc["status"] = payload.status.value
    doc["phase"] = payload.phase.value
    doc["created_at"] = doc["updated_at"] = now()
    try:
        doc["_id"] = trials.insert_one(doc).inserted_id
    except PyMongoError as e:
        raise HTTPException(status_code=500, detail=f"Database error: {e}")
    return to_out(doc)
```

Part 2: Memory : Short-, Long-, and Episodic Memory

This reuses the same mongodb database and adds three new collections as messages, summaries,episodes and a separate chat API



POST Request: The first chat message sent to the memory

Code:

```python
@app.post("/api/chat")
def chat(body: ChatIn):
    t0 = time.perf_counter()
    sid = body.session_id or str(uuid.uuid4())
    messages.insert_one({"user_id": body.user_id, "session_id": sid, "role": "user",
                         "content": body.message, "created_at": memory.now()})
    window = memory.short_term(body.user_id, sid, config.SHORT_TERM_N + 1)[:-1]
    sess = memory.latest_summary(body.user_id, "session", sid)
    life = memory.latest_summary(body.user_id, "user")
    facts = memory.retrieve_episodes(body.user_id, body.message, config.TOP_K_EPISODES)
    system, msgs = memory.build_prompt(sess, life, window, body.message, facts)

    resp = ollama.chat(model=config.CHAT_MODEL, messages=msgs, think=False)
    reply = resp["message"]["content"].strip()
    messages.insert_one({"user_id": body.user_id, "session_id": sid, "role": "assistant",
                         "content": reply, "created_at": memory.now()})

    memory.extract_episodes(body.user_id, sid, body.message)
    summarized = memory.maybe_summarize(body.user_id, sid)
    latency_ms = round((time.perf_counter() - t0) * 1000, 1)

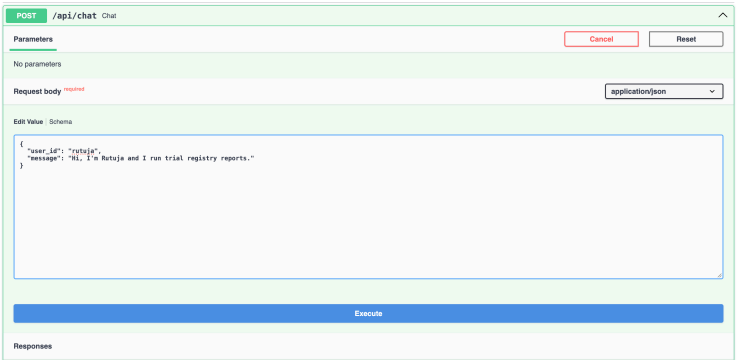
    return {
        "session_id": sid, "reply": reply,
        "memory_used": {
            "short_term_count": len(window), "short_term": window,
            "session_summary": sess, "lifetime_summary": life,
            "episodic_facts": facts,
        },
        "summary_updated": summarized,
        "metrics": {
            "prompt_tokens": resp.get("prompt_eval_count"),
            "completion_tokens": resp.get("eval_count"),
            "system_tokens_est": memory.count_tokens(config.SYSTEM_PRIMER),
            "long_term_tokens_est": memory.count_tokens(sess + life),
```

Code to Extract the episodes :

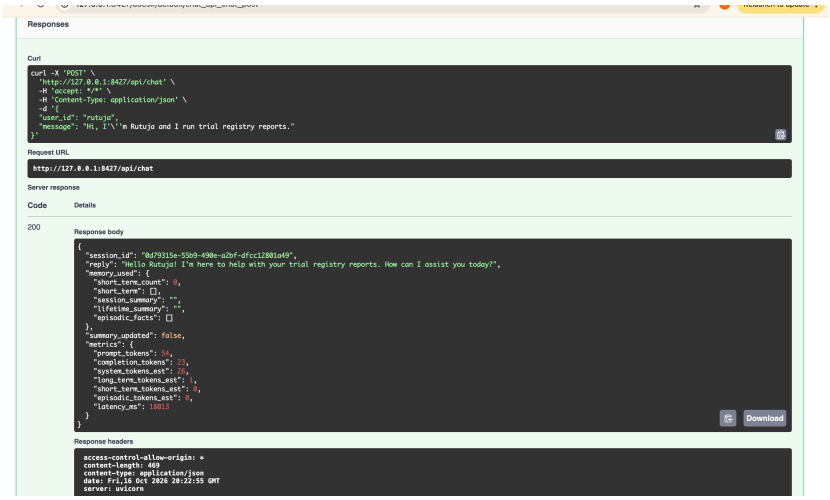
```python
# ---- episodic memory ----
def extract_episodes(user_id, session_id, user_msg):
    """Extract up to 3 short facts with an importance score (0..1)."""
    prompt = ("Extract up to 3 short facts worth remembering from this message. "
              'Reply ONLY with JSON: [{"fact": "...", "importance": 0.0-1.0}]. '
              "If nothing is worth remembering reply [].\n\nMessage: " + user_msg)
    raw, _ = llm(prompt)
    m = re.search(r"\[.*\]", raw, re.S)
    try:
        items = json.loads(m.group(0)) if m else []
    except json.JSONDecodeError:
        items = []
    saved = []
    for it in items[:3]:
        fact = str(it.get("fact", "")).strip()
        if not fact:
            continue
        imp = min(1.0, max(0.0, float(it.get("importance", 0.5))))
        doc = {"user_id": user_id, "session_id": session_id, "fact": fact,
               "importance": imp, "embedding": embed(fact), "created_at": now()}
        episodes.insert_one(doc)
        saved.append({"fact": fact, "importance": imp})
    return saved
```

Code to retrieve the episodes:

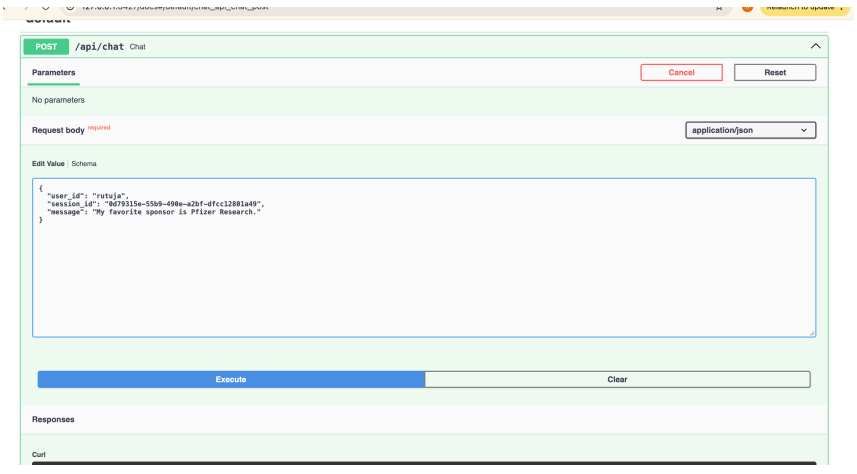
```python
def retrieve_episodes(user_id, message, k):
    """Top-k episodes by cosine similarity to the current message."""
    docs = list(episodes.find({"user_id": user_id}))
    if not docs:
        return []
    q = np.array(embed(message))
    scored = []
    for d in docs:
        e = np.array(d["embedding"])
        sim = float(np.dot(q, e) / (np.linalg.norm(q) * np.linalg.norm(e) + 1e-9))
        scored.append((sim, d["fact"]))
    scored.sort(reverse=True)
    return [{"fact": f, "similarity": round(s, 3)} for s, f in scored[:k]]
```



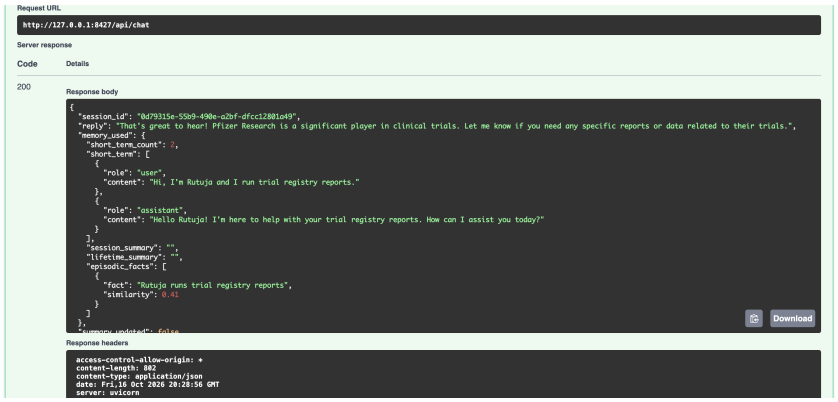
Response: Received the reply with the session id



POST Request: Sending another message with same session id, so the server loads the earlier messages (short-term memory). The fact ‘Rutuja runs trial registry reports’ was also retrieved (episodic memory).



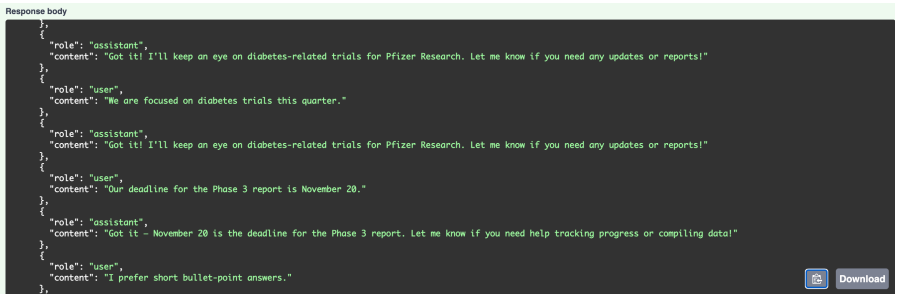
Response:




```
  "session_summary": ""
  "lifetime_summary": "",
  "episodic_facts": [
    {
      "fact": "Rutuja runs trial registry reports",
      "similarity": 0.41
    }
  ]
},
"summary_updated": false,
"metrics": {
  "prompt_tokens": 104,
  "completion_tokens": 33,
  "system_tokens_est": 26,
  "long_term_tokens_est": 1,
  "short_term_tokens_est": 35,
  "episodic_tokens_est": 8,
  "latency_ms": 3881.5
}
}
```


POST- Added few more messages with the same session id. A session summary now exists (long-term memory), the window is capped at 12 messages, and the saved preference made the reply use bullets.



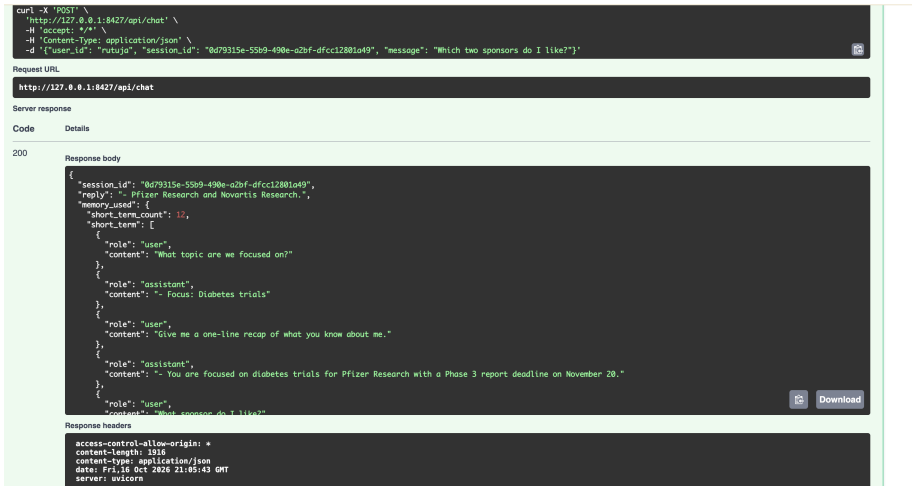




Code that builds the prompt:

```python
# ---- prompt assembly ----
def build_prompt(session_summary, lifetime_summary, window, message, facts):
    parts = [config.SYSTEM_PRIMER]
    if lifetime_summary:
        parts.append("Lifetime summary:\n" + lifetime_summary)
    if session_summary:
        parts.append("Session summary:\n" + session_summary)
    if facts:
        parts.append("Relevant facts:\n" + "\n".join("- " + f["fact"] for f in facts))
    system = "\n\n".join(parts)
    msgs = [{"role": "system", "content": system}] + window + [{"role": "user", "content": message}]
    return system, msgs
```

POST – The summary was updated again (summary_updated: true). The chatbot still keeps only the last 12 messages and remembered the new sponsor, Novartis Research.





Code that creates the summaries:

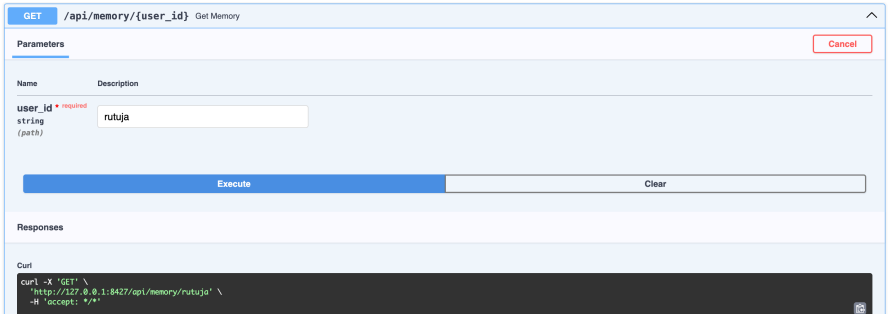
```python
def maybe_summarize(user_id, session_id):
    """Summarization trigger: every M user messages."""
    n_user = messages.count_documents({"user_id": user_id, "session_id": session_id, "role": "user"})
    if n_user == 0 or n_user % config.SUMMARIZE_EVERY_USER_MSGS != 0:
        return False
    recent = short_term(user_id, session_id, config.SUMMARIZE_EVERY_USER_MSGS * 2)
    convo = "\n".join(f"{m['role']}: {m['content']}" for m in recent)
    prev = latest_summary(user_id, "session", session_id)
    text, _ = llm(f"Previous summary:\n{prev}\n\nRecent conversation:\n{convo}\n\n"
                  "Write an updated summary as 3–5 concise bullets.")
    summaries.insert_one({"user_id": user_id, "session_id": session_id,
                          "scope": "session", "text": text, "created_at": now()})
    maybe_lifetime(user_id)
    return True
```

```python
def maybe_lifetime(user_id):
    """Condense several session summaries into one lifetime summary."""
    sess = list(summaries.find({"user_id": user_id, "scope": "session"}).sort("created_at", -1).limit(6))
    if len(sess) < config.LIFETIME_EVERY_N_SESSION_SUMMARIES:
        return
    joined = "\n---\n".join(s["text"] for s in reversed(sess))
    text, _ = llm(f"Session summaries:\n{joined}\n\nCondense into 3–5 bullets of lasting facts about this user.")
    summaries.insert_one({"user_id": user_id, "session_id": None,
                          "scope": "user", "text": text, "created_at": now()})
```

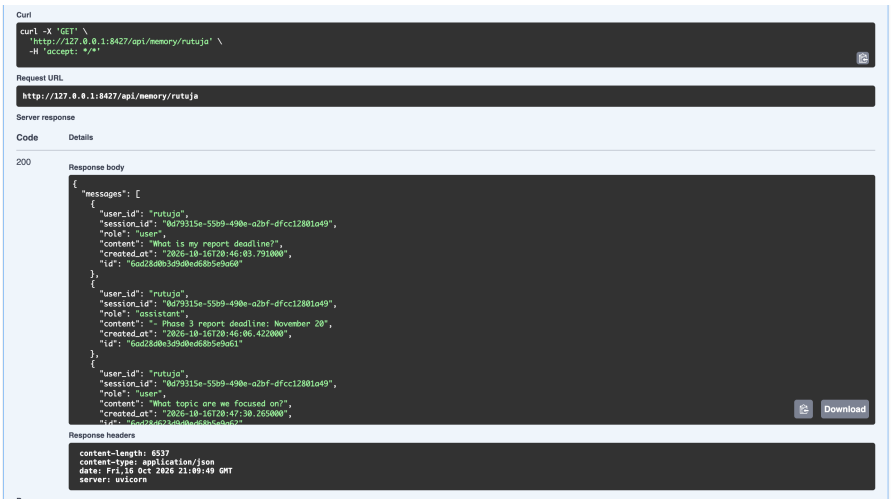
GET- Get memory using the user_id

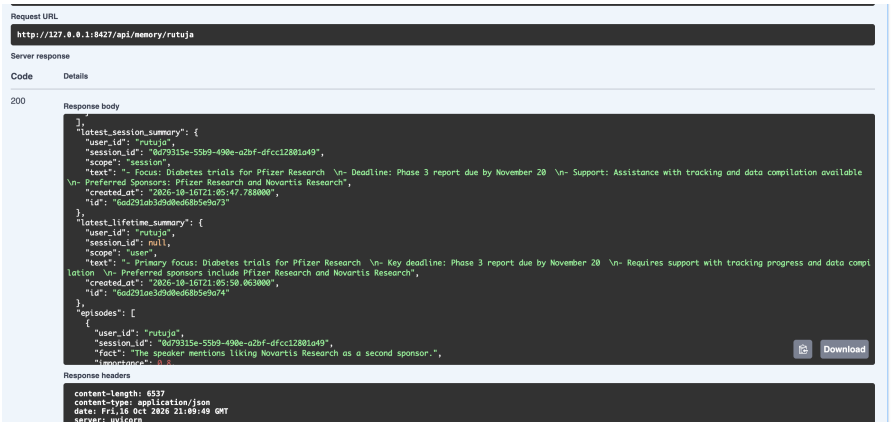
Code:

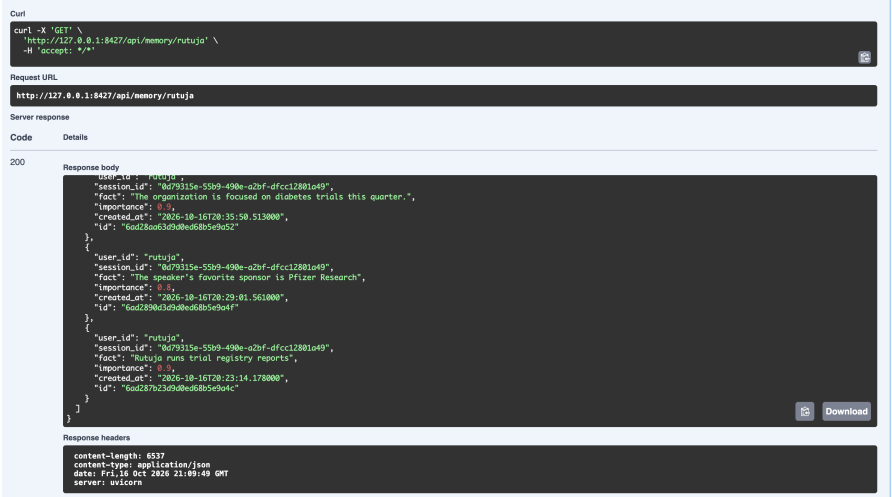
```python
@app.get("/api/memory/{user_id}")
def get_memory(user_id: str):
    msgs = [clean(d) for d in messages.find({"user_id": user_id}).sort("created_at", -1).limit(16)]
    sess = summaries.find_one({"user_id": user_id, "scope": "session"}, sort=[("created_at", -1)])
    life = summaries.find_one({"user_id": user_id, "scope": "user"}, sort=[("created_at", -1)])
    eps = [clean(d) for d in episodes.find({"user_id": user_id}).sort("created_at", -1).limit(20)]
    return {"messages": list(reversed(msgs)),
            "latest_session_summary": clean(sess) if sess else None,
            "latest_lifetime_summary": clean(life) if life else None,
            "episodes": eps}
```



Response - Sends the last 16 messages, the latest summary of the current chat, the lifetime summary built from several chats, and the saved facts (episodes), each with an importance score.





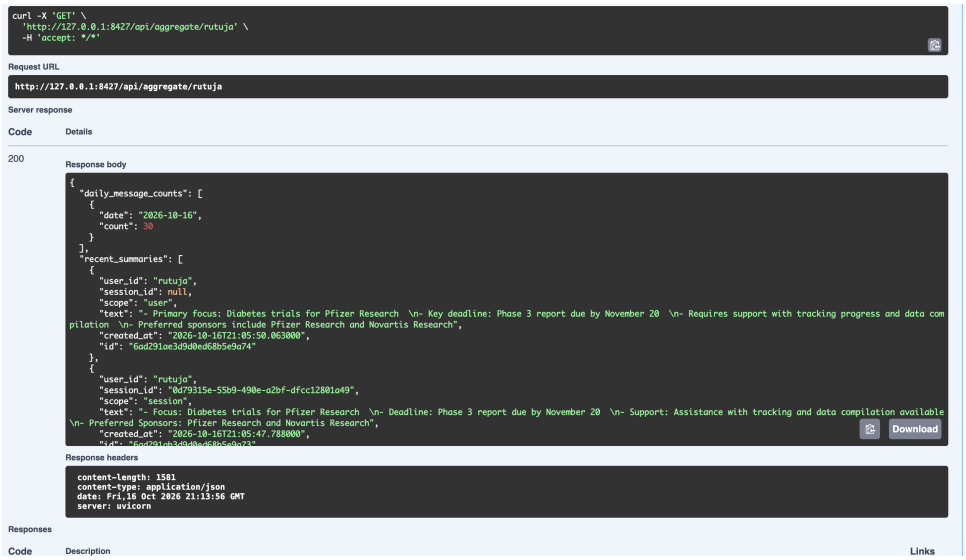


GET – Aggregate messages:

This groups the user's messages by date (30 messages today) and lists the most recent summaries: one lifetime summary and three session summaries.

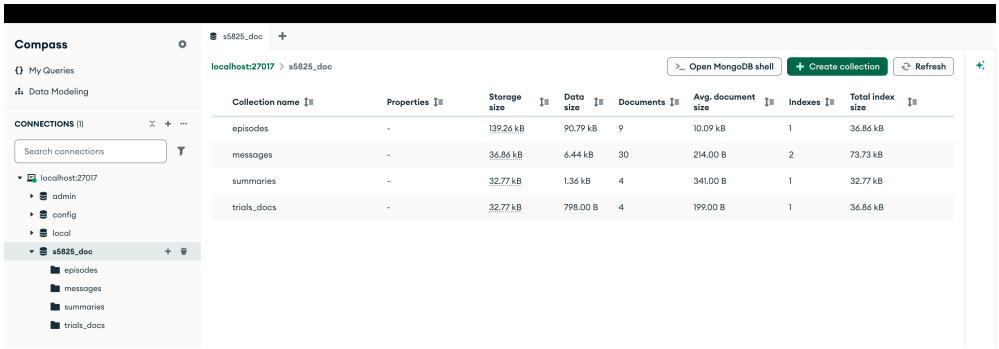
Code:

```python
@app.get("/api/aggregate/{user_id}")
def aggregate(user_id: str):
    daily = list(messages.aggregate([
        {"$match": {"user_id": user_id}},
        {"$group": {"_id": {"$dateToString": {"format": "%Y-%m-%d", "date": "$created_at"}},
                    "count": {"$sum": 1}}},
        {"$sort": {"_id": 1}}]))
    recent = [clean(d) for d in summaries.find({"user_id": user_id}).sort("created_at", -1).limit(5)]
    return {"daily_message_counts": [{"date": d["_id"], "count": d["count"]} for d in daily],
            "recent_summaries": recent}
```

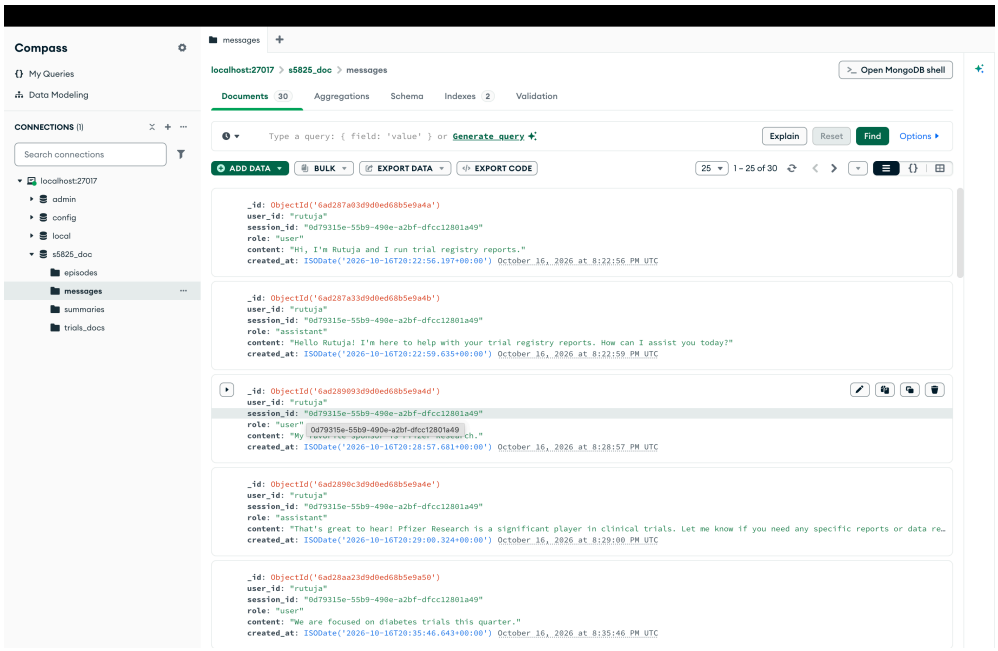


```
    "user_id": "rutuja",
    "session_id": "0d79315e-55b9-490e-a2bf-dfcc12801a49",
    "scope": "session",
    "text": "- Focus: Diabetes trials for Pfizer Research  \n- Deadline: Phase 3 report due by November 20  \n- Support: Assistance with tracking and data compilation available
\n- Preferred Sponsors: Pfizer Research and Novartis Research",
    "created_at": "2026-10-16T21:05:47.788000",
    "id": "6ad291ab3d9d0ed68b5e9a73"
  },
  {
    "user_id": "rutuja",
    "session_id": "0d79315e-55b9-490e-a2bf-dfcc12801a49",
    "scope": "session",
    "text": "- Focus: Diabetes trials for Pfizer Research  \n- Deadline: Phase 3 report due by November 20  \n- Support: Assistance with tracking and data compilation availabl
e",
    "created_at": "2026-10-16T20:50:57.819000",
    "id": "6ad28e313d9d0ed68b5e9a67"
  },
  {
    "user_id": "rutuja",
    "session_id": "0d79315e-55b9-490e-a2bf-dfcc12801a49",
    "scope": "session",
    "text": "- Rutuja is focused on diabetes trials for Pfizer Research this quarter.  \n- The Phase 3 report deadline is November 20.  \n- Assistance with tracking progress an
d compiling data for the report is available.",
    "created_at": "2026-10-16T20:36:16.206000",
    "id": "6ad28ac03d9d0ed68b5e9a59"
```

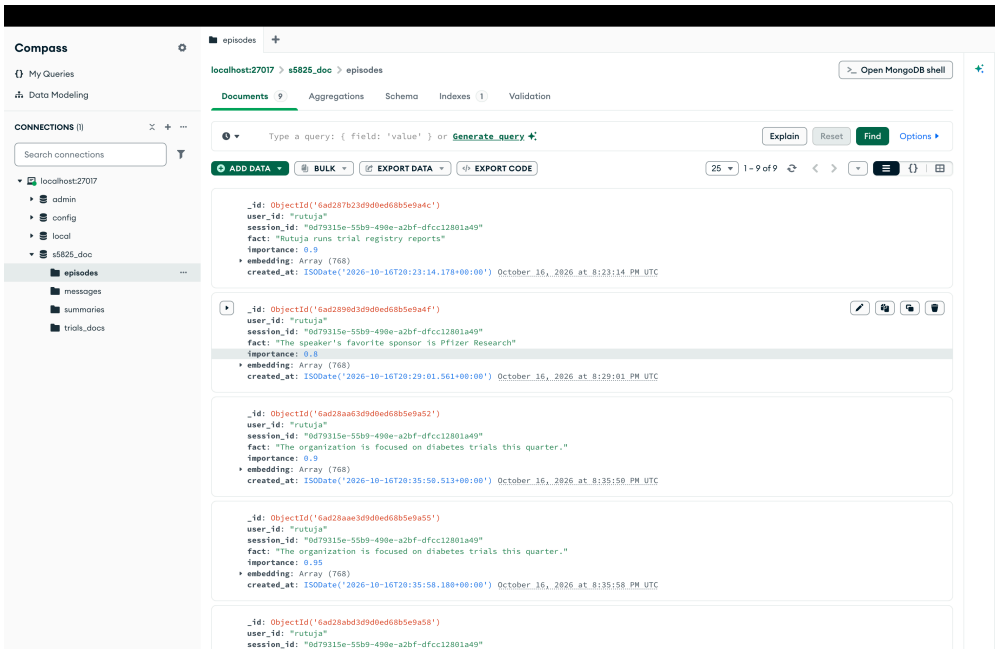

MongoDB Compass: s5825_doc holds the Part 1 collection (trials_docs) unchanged, plus the new messages, summaries and episodes collections.



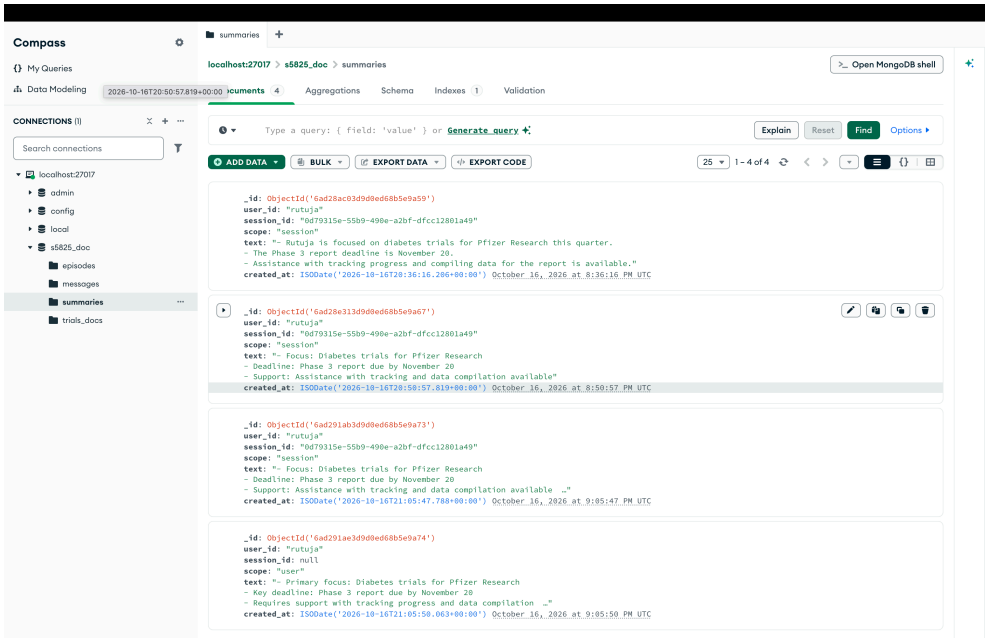
Documents stored in the “messages” collection:



Documents stored in episodes collection:

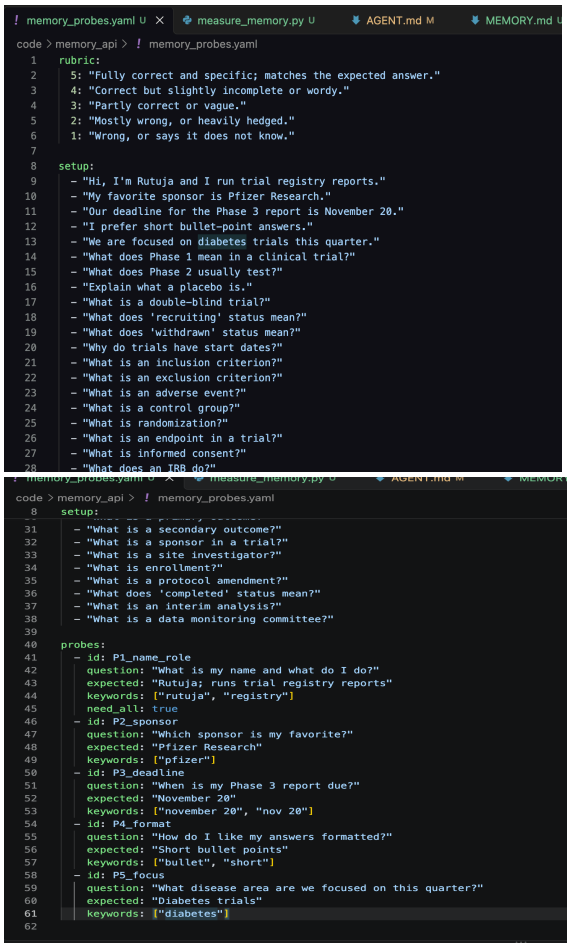


Documents stored in summaries collection:



Part 3 - What Memory Costs

The scoring guide (1–5) and the five fixed questions with their expected answers



```yaml
rubric:
  5: "Fully correct and specific; matches the expected answer."
  4: "Correct but slightly incomplete or wordy."
  3: "Partly correct or vague."
  2: "Mostly wrong, or heavily hedged."
  1: "Wrong, or says it does not know."

setup:
  - "Hi, I'm Rutuja and I run trial registry reports."
  - "My favorite sponsor is Pfizer Research."
  - "Our deadline for the Phase 3 report is November 20."
  - "I prefer short bullet-point answers."
  - "We are focused on diabetes trials this quarter."
  - "What does Phase 1 mean in a clinical trial?"
  - "What does Phase 2 usually test?"
  - "Explain what a placebo is."
  - "What is a double-blind trial?"
  - "What does 'recruiting' status mean?"
  - "What does 'withdrawn' status mean?"
  - "Why do trials have start dates?"
  - "What is an inclusion criterion?"
  - "What is an exclusion criterion?"
  - "What is an adverse event?"
  - "What is a control group?"
  - "What is randomization?"
  - "What is an endpoint in a trial?"
  - "What is informed consent?"
  - "What does an IRB do?"
  - "What is a secondary outcome?"
  - "What is a sponsor in a trial?"
  - "What is a site investigator?"
  - "What is enrollment?"
  - "What is a protocol amendment?"
  - "What does 'completed' status mean?"
  - "What is an interim analysis?"
  - "What is a data monitoring committee?"

probes:
  - id: P1_name_role
    question: "What is my name and what do I do?"
    expected: "Rutuja; runs trial registry reports"
    keywords: ["rutuja", "registry"]
    need_all: true
  - id: P2_sponsor
    question: "Which sponsor is my favorite?"
    expected: "Pfizer Research"
    keywords: ["pfizer"]
  - id: P3_deadline
    question: "When is my Phase 3 report due?"
    expected: "November 20"
    keywords: ["november 20", "nov 20"]
  - id: P4_format
    question: "How do I like my answers formatted?"
    expected: "Short bullet points"
    keywords: ["bullet", "short"]
  - id: P5_focus
    question: "What disease area are we focused on this quarter?"
    expected: "Diabetes trials"
    keywords: ["diabetes"]
```

Each run sets SHORT_TERM_N, creates a brand-new user and session id, and clears old data first, so the three runs never share memory.

```python
def run(label, n, spec, quick, policy=False):
    config.SHORT_TERM_N = n
    memory.short_term = compressed_short_term if policy else _orig_short_term
    uid, sid = f"probe_{label}", str(uuid.uuid4())
    clean(uid)
    setup = spec["setup"][:8] if quick else spec["setup"]
    turns, probes = [], []
    log(f"START {label} N={n} policy={policy} user={uid} session={sid}")
```

Memory Cost Results (30-message conversation, 5 probe question):

Setting	Mean prompt tokens/turn	Mean latency (ms)	Probe quality (1-5)
SHORT_TERM_N=4	243.4	4461.1	5.0
SHORT_TERM_N=12	404.1	5104.8	4.4
SHORT_TERM_N=30	687.5	5664.9	4.4

Setting	Mean prompt tokens/turn	Mean latency (ms)	Probe quality (1-5)
Eviction/compression policy	365.0	4889.8	4.4

Estimated Cost for 10,000 Conversations of 30 Turns:

Setting	Mean prompt tokens/turn	Mean completion tokens/turn	Estimated cost
SHORT_TERM_N=4 (best)	243.4	23.9	$108.89
SHORT_TERM_N=12	404.1	25.9	$160.09
SHORT_TERM_N=30 (worst)	687.5	27.1	$246.87
Eviction/compression policy	365.0	24.3	$145.92

Explain how you selected the best and worst settings.

Selected the worst setting as the one that costs the most: SHORT_TERM_N=30 ($246.87).

Selected the best setting as the cheapest one whose answer quality was close to the top score (within 0.5 points): SHORT_TERM_N=4 ($108.89). These costs only count the chat messages, not the extra calls that write summaries and save facts.

Eviction/Compression Policy Result:

Policy: the chatbot reads the last 12 messages, but only the 5 newest are kept in full. Older messages are shortened to their first 80 characters. Compared with normal N=12, this sent about 10% fewer tokens (404.1 to 365.0 per turn) and was a little faster (about 215 ms). The answer quality stayed the same (4.4 in both).

```
def compressed_short_term(user_id, session_id, n):
    w = _orig_short_term(user_id, session_id, n)
    out = []
    for i, m in enumerate(w):
        m = dict(m)
        if i < len(w) - KEEP_FULL and len(m["content"]) > TRUNC:
            m["content"] = m["content"][:TRUNC] + "..."
        out.append(m)
    return out
```

Fresh Session Check: Each run used its own user and session ID, so no memory was shared between runs.

```
rutujasaste@Rutujas-MacBook-Pro data260-5825 %  source /Users/rutujasaste/Documents/data260-5825/venv/bin/activate
(venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 % mongosh s5825_doc --quiet --eval 'db.messages.aggregate([{$match:{user_id:/^probe_/}},{$g
roup:{_id:{user:"$user_id",session:"$session_id"},msgs:{$sum:1}}}]).forEach(printjson)'
{
  _id: {
    user: 'probe_n30',
    session: '925c56be-ebc6-470e-9f6e-0925a3d990c9'
  },
  msgs: 70
}
{
  _id: {
    user: 'probe_n12',
    session: '572a465b-ae8c-4a9c-9431-7274bd3ec504'
  },
  msgs: 70
}
{
  _id: {
    user: 'probe_n4',
    session: 'b10695e1-38ed-4d75-a339-160414e9b983'
  },
  msgs: 70
}
{
  _id: {
    user: 'probe_policy',
    session: 'f21de355-3e48-4198-8e14-50e3ace39d22'
  },
  msgs: 70
}
```

Part 4 - File-Based Memory and Consolidation:

Function Check Against AGENT.md:

```
de > file_memory > trial_utils.py > format_trial_label
1   def format_trial_label(title: str, phase: str) -> str:
2       """Return a short label like 'Metformin for Diabetes (phase2)'."""
3       return f"{title.strip()[:100]} ({phase.strip().lower()})"
```

The function format_trial_label follows AGENT.md: type hints, a one-line docstring, snake_case, under 20 lines, no print(), no secrets, and a pytest test.

```
(venv) rutujasaste@Rutujas-MacBook-Pro file_memory % clear
python -m pytest tests
python verify_agent.py
============================= test session starts ==============================
platform darwin -- Python 3.11.17, pytest-9.1.1, pluggy-1.7.0
rootdir: /Users/rutujasaste/Documents/data260-5825/code/file_memory
plugins: langsmith-0.12.3, anyio-4.15.1
collected 1 item

tests/test_trial_utils.py .                                              [100%]

============================== 1 passed in 0.00s ===============================
Rutuja Saste - checking trial_utils.py against AGENT.md
[PASS] has at least one function
[PASS] type hints on all args and return
[PASS] one-line docstring
[PASS] snake_case names
[PASS] under 20 lines
[PASS] no print()
[PASS] no hard-coded secrets
[PASS] pytest test exists
(venv) rutujasaste@Rutujas-MacBook-Pro file_memory %
```

Memory Commands: /help

The chat program starts with a memory check (55 lines is more than 40, so consolidation is needed) and /help lists the four memory commands.

```
(venv) rutujasaste@Rutujas-MacBook-Pro file_memory % cd ~/Documents/data260-5825
git add code/file_memory/MEMORY.md code/file_memory/MEMORY_original.md
git commit -m "HW6 Part 4: original 55-line MEMORY.md (before consolidation)"
[main a7761e6] HW6 Part 4: original 55-line MEMORY.md (before consolidation)
 2 files changed, 110 insertions(+)
 create mode 100644 code/file_memory/MEMORY.md
 create mode 100644 code/file_memory/MEMORY_original.md
(venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 % cd ~/Documents/data260-5825/code/file_memory
source ~/Documents/data260-5825/venv/bin/activate
python chat_cli.py
File-memory chat - Rutuja Saste. Type /help for commands.
[memory check] consolidation needed: True (55 lines > 40)
you> /help
Commands:
/memory   show AGENT.md and MEMORY.md
/compact  consolidate MEMORY.md and shrink the chat (prints counts before/after)
/dream    save useful long-term facts from the recent chat into MEMORY.md
/help     show this help
/quit     exit
you>
```

Code:

```
HELP = """Commands:
/memory   show AGENT.md and MEMORY.md
/compact  consolidate MEMORY.md and shrink the chat (prints counts before/after)
/dream    save useful long-term facts from the recent chat into MEMORY.md
/help     show this help
/quit     exit"""
```

Memory Commands:/memory:

```
(venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 % cd ~/Documents/data260-5825/code/file_memory
source ~/Documents/data260-5825/venv/bin/activate
python chat_cli.py
you> /memory
=== AGENT.md ===
# Agent Instructions

## Code review responses
- When asked to review code, you MUST respond using ONLY bullet points.
- Do not include any introductory sentences, paragraphs, explanations outside of bullets, or concluding remarks. Every piece of feedback must b
e a single bullet point starting with "- ".
- Do not use headings, code blocks, or numbered lists in review responses. Bullet points only.

## Project description
- This project is a Clinical Trial Listings system: FastAPI + MongoDB APIs and a chat assistant with memory.
- The student is Rutuja Saste (SID4 5825). Explain things in simple words.

## Technology stack
- Use Python 3.11, FastAPI, PyMongo and Pydantic.
- The local model is qwen3:8b through Ollama. Do not call paid hosted models.

## Coding conventions
- Every function must have type hints and a one-line docstring.
- Use snake_case names and keep each function under 20 lines.
- Application functions must not use print(). Use logging. Only the chat CLI may print.

## Testing approach
- Every new function needs at least one pytest test in a tests/ folder.

## Prohibited actions
- Never hard-code passwords, API keys or database credentials.
- Do not edit code/doc_api or code/memory_api unless asked.
=== MEMORY.md ===
- (2026-09-01) User name is Rutuja Saste and studies at SJSU.
- (2026-09-01) Course is DATA 260 in the MSADI program.
- (2026-09-02) Project domain is Clinical Trial Listings.
- (2026-09-02) SID4 is 5825.
- (2026-09-02) The repository is data260-5825 on GitHub.
- (2026-09-03) The API runs on port 8425.
- (2026-09-03) Favorite sponsor is Pfizer Research.
- (2026-09-03) Phase 3 report deadline is November 20.
- (2026-09-04) User prefers short bullet-point answers.
- (2026-09-04) Backend uses Python 3.11 and FastAPI.
- (2026-09-05) The database is MongoDB.
- (2026-09-05) The local model is qwen3:8b via Ollama.
- (2026-09-06) Tests are written with pytest.
- (2026-09-06) Collaborators are Sbnikitha and supriyaselvanganesan.
- (2026-09-07) The database name is s5825_doc.
- (2026-09-07) The Part 1 collection is trials_docs.
- (2026-09-08) Memory collections are messages, summaries and episodes.
- (2026-09-08) The short-term window default is 12 messages.
- (2026-09-09) A session summary is created every 5 user messages.
- (2026-09-09) Episodic retrieval returns the top 3 facts.
- (2026-09-10) User works on a Mac with an M5 chip.
- (2026-09-10) The virtual environment is in the venv folder.
- (2026-09-11) The embedding model is nomic-embed-text.
- (2026-09-11) User wants simple explanations in plain words.
- (2026-09-12) Never commit passwords to Git.
- (2026-09-12) Homework 5 was tagged hw5.
```

```
- (2026-09-12) Homework 5 was tagged hw5.
- (2026-09-13) Trial statuses are recruiting, active, completed and withdrawn.
- (2026-09-13) Trial phases are phase1 to phase4.
- (2026-09-14) Dates are stored as ISODate in MongoDB.
- (2026-09-14) The memory API runs on port 8427.
- (2026-09-15) Pfizer Research is the sponsor the user likes most.
- (2026-09-15) The Phase 3 report is due on November 20.
- (2026-09-16) Answers should be short bullet points.
- (2026-09-16) The backend is written in Python 3.11 with FastAPI.
- (2026-09-17) The API port is 8425.
- (2026-09-17) Pfizer Research is the favorite sponsor.
- (2026-09-18) November 20 is the deadline for the Phase 3 report.
- (2026-09-18) User likes short answers in bullets.
- (2026-09-19) The model used locally is qwen3:8b.
- (2026-09-19) User uses a MacBook with an M5 chip.
- (2026-09-20) The data lives in MongoDB.
- (2026-09-20) The default short-term window is 12 messages.
- (2026-09-21) Session summaries run every 5 user messages.
- (2026-09-21) Top 3 episodic facts are retrieved.
- (2026-09-22) Embeddings come from nomic-embed-text.
- (2026-09-22) Passwords must never be committed to Git.
- (2026-09-23) The repo is data260-5825.
- (2026-09-23) The memory API listens on port 8427.
- (2026-09-24) Explanations should be simple and in plain words.
- (2026-09-24) The database is named s5825_doc.
- (2026-09-25) Tests use pytest.
- (2026-10-01) The Phase 3 report deadline moved to December 5 (confirmed by the sponsor).
- (2026-10-02) Favorite sponsor is now Novartis Research.
- (2026-10-03) User now prefers long detailed paragraph answers.
- (2026-10-04) The API runs on port 8000.
you>
```

Code:

```python
def load_memory_files():
    return read_file(AGENT_PATH), read_file(MEMORY_PATH)


def build_system_prompt():
    agent, memory = load_memory_files()
    return ("You are a helpful assistant for a clinical trial listings project.\n\n"
            "## AGENT.md (standing instructions)\n" + agent +
            "\n\n## MEMORY.md (facts learned earlier)\n" + memory)
```

/compact command. The chat history shrank from 6 messages to 1. MEMORY.md stayed at 35 lines and the Resolved contradictions section was kept

```
(venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 % cd ~/Documents/data260-5825
git checkout -- code/file_memory/MEMORY.md code/file_memory/dedupe_log.txt
(venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 % cd ~/Documents/data260-5825/code/file_memory
python chat_cli.py
File-memory chat - Rutuja Saste. Type /help for commands.
[memory check] consolidation needed: False (35 lines, last consolidated 0.5 h ago)
you> My name is Rutuja and I am testing file memory.
bot> - Your name is Rutuja Saste.
you> Please remember that I prefer examples in Python.
bot> - Got it! I'll provide examples in Python when explaining concepts.
you> What is the Phase 3 report deadline?
bot> - The Phase 3 report deadline is December 5.
you> /compact
[compact] chat messages: 6 -> 1 | MEMORY.md lines: 35 -> 35
you>
```

Code:

```python
def compact(history):
    """Consolidate MEMORY.md and shrink the chat history to one summary message."""
    msgs_before = len(history)
    lines_before, lines_after = consolidate()
    if history:
        summary = llm("Summarize this chat in 3 short bullets:\n" + _convo(history, 50))
        history[:] = [{"role": "assistant", "content": "Summary of earlier chat:\n" + summary}]
    return msgs_before, len(history), lines_before, lines_after
```

/dream command. It picked the two lasting facts from the chat and appended them to MEMORY.md with today's date.

```
(venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 % cd ~/Documents/data260-5825/code/file_memory
python chat_cli.py
File-memory chat - Rutuja Saste. Type /help for commands.
[memory check] consolidation needed: False (35 lines, last consolidated 0.1 h ago)
you> Remember that my final HW6 demo uses the data260-5825 repository.
bot> - Your final HW6 demo uses the data260-5825 repository.
you> I decided to name the report file Saste_HW6.pdf.
bot> - You decided to name the report file Saste_HW6.pdf.
you> /dream
[dream] added:
  - (2026-10-18) Your final HW6 demo uses the data260-5825 repository.
  - (2026-10-18) You decided to name the report file Saste_HW6.pdf.
you> /quit
```

Code:

```python
def dream(history):
    """Pick durable long-term facts from the recent chat and append them to MEMORY.md."""
    if not history:
        return []
    out = llm("From this conversation list up to 5 durable long-term facts about the "
              "user or project (names, preferences, deadlines, decisions). Skip small "
              "talk. One per line, each starting with '- '. If none, reply NONE.\n\n"
              + _convo(history))
    existing = read_file(MEMORY_PATH)
    today = now_utc().date().isoformat()
    added = []
    for ln in out.splitlines():
        ln = ln.strip()
        if ln.startswith("- ") and ln[2:].lower() not in existing.lower():
            added.append(f"- ({today}) {ln[2:]}")
    if added:
        write_file(MEMORY_PATH, existing.rstrip("\n") + "\n" + "\n".join(added) + "\n")
    return added
```

Consolidation Test: 55 lines to 35 lines:

Created a 55-line MEMORY.md with 21 reworded duplicate lines and 4 deliberate contradictions. The consolidation ran automatically before my next chat message and reduced it to 35 lines.

```
(venv) rutujasaste@Rutujas-MacBook-Pro file_memory % cd ~/Documents/data260-5825/code/file_memory
cp MEMORY_original.md MEMORY.md
rm -f dedupe_log.txt
python chat_cli.py
File-memory chat - Rutuja Saste. Type /help for commands.
[memory check] consolidation needed: True (55 lines > 40)
you> Which sponsor do I like?
[auto-consolidate] 55 lines > 40: 55 -> 35 lines
bot> - Your favorite sponsor is Novartis Research.
you> /quit
```

Code:



```python
# ---------- consolidation checks ----------
def count_memory_lines(text):
    """Non-empty lines that are not <!-- comments -->."""
    return sum(1 for ln in text.splitlines()
               if ln.strip() and not ln.strip().startswith("<!--"))


def read_last_consolidated(text=None):
    text = read_file(MEMORY_PATH) if text is None else text
    m = STAMP_RE.search(text)
    if not m:
        return None
    try:
        t = dt.datetime.fromisoformat(m.group(1))
    except ValueError:
        return None
    return t if t.tzinfo else t.replace(tzinfo=dt.timezone.utc)


def set_last_consolidated(text, when=None):
    stamp = f"<!-- last_consolidated: {(when or now_utc()).isoformat()} -->"
    if STAMP_RE.search(text):
        return STAMP_RE.sub(lambda m: stamp, text, count=1)
    return stamp + "\n" + text


def needs_consolidation():
    text = read_file(MEMORY_PATH)
    n = count_memory_lines(text)
    if n == 0:
        return False, "MEMORY.md is empty"
    if n > MAX_LINES:
        return True, f"{n} lines > {MAX_LINES}"
    last = read_last_consolidated(text)
    if last is None:
        return True, "no consolidation time recorded"
    age = (now_utc() - last).total_seconds() / 3600
    if age > MAX_AGE_HOURS:
        return True, f"last consolidated {age:.1f} h ago (> {MAX_AGE_HOURS} h)"
    return False, f"{n} lines, last consolidated {age:.1f} h ago"
```

Original MEMORY.md (55 lines):

Lines 31 to 51 repeat earlier facts in different words. Lines 52 to 55 contain the four planted contradictions: Phase 3 deadline (November 20 vs December 5), favorite sponsor (Pfizer Research vs Novartis Research), answer style (short bullets vs long paragraphs) and API port (8425 vs 8000).

MEMORY_original_55_lines.md

reports > hw06 > part4 > MEMORY_original_55_lines.md

```
- (2026-09-01) User name is Rutuja Saste and studies at SJSU.
- (2026-09-01) Course is DATA 260 in the MSADI program.
- (2026-09-02) Project domain is Clinical Trial Listings.
- (2026-09-02) SID4 is 5825.
- (2026-09-02) The repository is data260-5825 on GitHub.
- (2026-09-03) The API runs on port 8425.
- (2026-09-03) Favorite sponsor is Pfizer Research.
- (2026-09-03) Phase 3 report deadline is November 20.
- (2026-09-04) User prefers short bullet-point answers.
- (2026-09-04) Backend uses Python 3.11 and FastAPI.
- (2026-09-05) The database is MongoDB.
- (2026-09-05) The local model is qwen3:8b via Ollama.
- (2026-09-06) Tests are written with pytest.
- (2026-09-06) Collaborators are Sbnikitha and supriyaselvanganesan.
- (2026-09-07) The database name is s5825_doc.
- (2026-09-07) The Part 1 collection is trials_docs.
- (2026-09-08) Memory collections are messages, summaries and episodes.
- (2026-09-08) The short-term window default is 12 messages.
- (2026-09-09) A session summary is created every 5 user messages.
- (2026-09-09) Episodic retrieval returns the top 3 facts.
- (2026-09-10) User works on a Mac with an M5 chip.
- (2026-09-10) The virtual environment is in the venv folder.
- (2026-09-11) The embedding model is nomic-embed-text.
- (2026-09-11) User wants simple explanations in plain words.
- (2026-09-12) Never commit passwords to Git.
- (2026-09-12) Homework 5 was tagged hw5.
- (2026-09-13) Trial statuses are recruiting, active, completed and withdrawn.
- (2026-09-13) Trial phases are phase1 to phase4.
- (2026-09-14) Dates are stored as ISODate in MongoDB.
- (2026-09-14) The memory API runs on port 8427.
- (2026-09-15) Pfizer Research is the sponsor the user likes most.
- (2026-09-15) The Phase 3 report is due on November 20.
- (2026-09-16) Answers should be short bullet points.
- (2026-09-16) The backend is written in Python 3.11 with FastAPI.
- (2026-09-17) The API port is 8425.
- (2026-09-17) Pfizer Research is the favorite sponsor.
- (2026-09-18) November 20 is the deadline for the Phase 3 report.
- (2026-09-18) User likes short answers in bullets.
- (2026-09-19) The model used locally is qwen3:8b.
- (2026-09-19) User uses a MacBook with an M5 chip.
- (2026-09-20) The data lives in MongoDB.
- (2026-09-20) The default short-term window is 12 messages.
- (2026-09-21) Session summaries run every 5 user messages.
- (2026-09-21) Top 3 episodic facts are retrieved.
- (2026-09-22) Embeddings come from nomic-embed-text.
- (2026-09-22) Passwords must never be committed to Git.
```

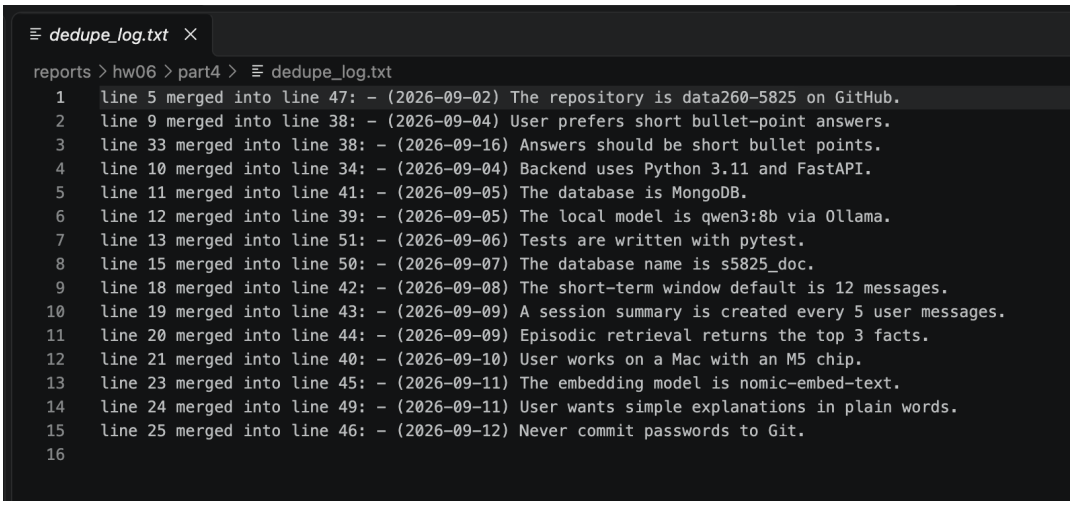
Rutuja Saste (1 day ago) Ln 26, Col 42 Spaces: 4 UTF

```
- (2026-09-12) Homework 5 was tagged hw5.
- (2026-09-13) Trial statuses are recruiting, active, completed and withdrawn.
- (2026-09-13) Trial phases are phase1 to phase4.
- (2026-09-14) Dates are stored as ISODate in MongoDB.
- (2026-09-14) The memory API runs on port 8427.
- (2026-09-15) Pfizer Research is the sponsor the user likes most.
- (2026-09-15) The Phase 3 report is due on November 20.
- (2026-09-16) Answers should be short bullet points.
- (2026-09-16) The backend is written in Python 3.11 with FastAPI.
- (2026-09-17) The API port is 8425.
- (2026-09-17) Pfizer Research is the favorite sponsor.
- (2026-09-18) November 20 is the deadline for the Phase 3 report.
- (2026-09-18) User likes short answers in bullets.
- (2026-09-19) The model used locally is qwen3:8b.
- (2026-09-19) User uses a MacBook with an M5 chip.
- (2026-09-20) The data lives in MongoDB.
- (2026-09-20) The default short-term window is 12 messages.
- (2026-09-21) Session summaries run every 5 user messages.
- (2026-09-21) Top 3 episodic facts are retrieved.
- (2026-09-22) Embeddings come from nomic-embed-text.
- (2026-09-22) Passwords must never be committed to Git.
- (2026-09-23) The repo is data260-5825.
- (2026-09-23) The memory API listens on port 8427.
- (2026-09-24) Explanations should be simple and in plain words.
- (2026-09-24) The database is named s5825_doc.
- (2026-09-25) Tests use pytest.
- (2026-10-01) The Phase 3 report deadline moved to December 5 (confirmed by the sponsor).
- (2026-10-02) Favorite sponsor is now Novartis Research.
- (2026-10-03) User now prefers long detailed paragraph answers.
- (2026-10-04) The API runs on port 8000.
```

Resolved incorrectly: the API port. The newest-date rule kept 'The API runs on port 8000', but the correct port is 8425. The memory API port (8427) was also wrongly grouped into the same conflict, although it is still listed in the REPLACED part. Not detected: short bullet answers vs long paragraph answers. Both lines are still in the file.

Duplicate Log

Every merged duplicate is listed with the line it was merged into, so no fact was removed without a record.



MongoDB Memory vs File-Based Memory:

	MongoDB memory (Part 2)	File memory (Part 4)
Inspectability	Needs a query or Compass to see what is stored. Embeddings are numbers a person cannot read.	Open MEMORY.md in any editor and read every fact. git diff shows exactly what changed.
Persistence after restart	Data stays in the database. After an API restart the messages, summaries and facts are still there.	Data stays in the files. After I restarted the chat program it still answered December 5 and Novartis Research from MEMORY.md.
Scalability across many sessions	Good. It can hold very many messages and facts, and each turn retrieves only the top 3 relevant facts, so the prompt stays small.	Poor. The whole file goes into every prompt, so it gets slower and more expensive as it grows. It needs consolidation to stay under 40 lines.
Facts that must not be forgotten	Risky. A fact is used only if it ranks in the top 3 for that question, so an important fact can be missed.	Safer. AGENT.md and MEMORY.md are loaded in full on every turn, so a rule such as never commit passwords is always in front of the model.

AI_USE:

1. What I used AI for, and what I did myself

I used AI to explain each part of the homework in simple words and also used it for implementation of the memory API, the Part 3 measurement script, and the file-memory chat program. I created the mongo db API by understanding previous assignment API implementations also took help of AI but tried at my level for better understanding, validated MongoDB, tested all commands. I also wrote the AGENT.md rules and chose the planted duplicates and contradictions in MEMORY.md.

2. One AI output that was wrong or unsuitable

The AI's first consolidation design asked qwen3:8b to rewrite the whole 55-line MEMORY.md. It only reached 52 lines, which fails the "under 40 lines" rule. A second design asked the model to group line numbers. It reached 29 lines, but it grouped unrelated lines so real facts were lost, and the contradictions were not resolved.

3. How I detected the problem

I counted lines with "wc -l" after each run and read the full MEMORY.md and dedupe_log.txt line by line. The 52-line result missed the line target. In the 29-line result I saw both Pfizer and Novartis still in the file and unrelated facts in the log. Later, "git diff" after "/compact" command showed that a second consolidation deleted "Resolved contradictions" lines.

4. What I changed and why it works now

I divided the task into smaller steps. First, the embedding model finds lines that may be similar. Then, the model checked each pair and labeled it as SAME, CONFLICT, or DIFFERENT. Python then kept the line with the newest date and saves all removed lines in a log. I also changed the consolidation code so it skips the “Resolved contradictions” section and adds new information to the log instead of replacing it. After these changes, MEMORY.md went from 55 lines to 35 lines. When I ran "/compact" again, "git diff" showed that only the timestamp had changed.

There are still some limitations. The port 8000 contradiction was resolved incorrectly because the newer line was not accurate, the correct API port is 8425. The system also did not detect the contradiction between using short bullets and long paragraphs.